3.3、图像阈值化

[OpenCV: Image Thresholding]
https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

图像阈值化是图像处理中最基础、最常用的操作之一，它把灰度图像转换成二值图像（黑白图像），常用于分割前景和背景、文字识别、物体检测等。

1. 简单阈值化（Simple Thresholding）

核心思想：对整张图像使用同一个固定阈值（threshold）来处理每个像素。

- 如果像素值 > 阈值 → 设置为最大值（通常是 255，白色）
- 如果像素值 ≤ 阈值 → 设置为 0（黑色）
  OpenCV 使用函数：

```
ret, dst = cv.threshold(src, thresh, maxval, type)
```

- src：必须是灰度图像（单通道）
- thresh：阈值（例如 127）
- maxval：超过阈值时赋予的值（通常 255）
- type：阈值化类型（最常用下面几种）
- ret：实际使用的阈值（后面会用到）

5 种常见的阈值化类型

1. cv.THRESH_BINARY（最常用）
   thresh 时 → 255，否则 → 0
2. cv.THRESH_BINARY_INV（反转）
   thresh 时 → 0，否则 → 255
3. cv.THRESH_TRUNC（截断）
   thresh 时 → 阈值本身（127），否则保持原值
4. cv.THRESH_TOZERO
   thresh 时 → 0，否则保持原值
5. cv.THRESH_TOZERO_INV
   thresh 时 → 保持原值，否则 → 0

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt
```

```
# 读取灰度图（官方示例用 gradient.png，你可以用任意渐变灰度图）
img = cv.imread('gradient.png', cv.IMREAD_GRAYSCALE)
assert img is not None, "文件读取失败，请检查路径"

ret, thresh1 = cv.threshold(img, 127, 255, cv.THRESH_BINARY)
ret, thresh2 = cv.threshold(img, 127, 255, cv.THRESH_BINARY_INV)
ret, thresh3 = cv.threshold(img, 127, 255, cv.THRESH_TRUNC)
ret, thresh4 = cv.threshold(img, 127, 255, cv.THRESH_TOZERO)
ret, thresh5 = cv.threshold(img, 127, 255, cv.THRESH_TOZERO_INV)

titles = ['Original Image', 'BINARY', 'BINARY_INV', 'TRUNC', 'TOZERO',
'TOZERO_INV']
images = [img, thresh1, thresh2, thresh3, thresh4, thresh5]

for i in range(6):
    plt.subplot(2, 3, i+1)
    plt.imshow(images[i], 'gray')
    plt.title(titles[i])
    plt.xticks([]), plt.yticks([])
plt.show()
```

效果：从左到右依次看到原始图和 5 种不同处理结果。你会明显看到 BINARY 把图像变成清晰的黑白，TRUNC 把亮的部分“压平”到阈值等。

2. 自适应阈值化（Adaptive Thresholding）

问题：简单阈值化对光照不均匀的图像效果很差（例如照片里有阴影）。
解决办法：每个像素的阈值不是固定的，而是根据它周围小区域的亮度来计算。OpenCV 函数：

```
th = cv.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType,
blockSize, C)
```

参数说明：

- adaptiveMethod：
  - cv.ADAPTIVE_THRESH_MEAN_C：阈值 = 邻域平均值 - C
  - cv.ADAPTIVE_THRESH_GAUSSIAN_C：阈值 = 邻域高斯加权平均 - C
- blockSize：计算阈值的邻域大小，必须是奇数（如 11、21），因为要以当前像素为中心
- C：常数，通常取 2~10，用于微调
  推荐先做中值滤波去噪：cv.medianBlur(img, 5)

```
img = cv.imread('sudoku.png', cv.IMREAD_GRAYSCALE)
img = cv.medianBlur(img, 5)          # 去噪很重要

ret, th1 = cv.threshold(img, 127, 255, cv.THRESH_BINARY)                    #
全局阈值（对比用）
th2 = cv.adaptiveThreshold(img, 255, cv.ADAPTIVE_THRESH_MEAN_C,
                           cv.THRESH_BINARY, 11, 2)
th3 = cv.adaptiveThreshold(img, 255, cv.ADAPTIVE_THRESH_GAUSSIAN_C,
                           cv.THRESH_BINARY, 11, 2)

titles = ['Original', 'Global (127)', 'Adaptive Mean', 'Adaptive Gaussian']
images = [img, th1, th2, th3]

for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.imshow(images[i], 'gray')
    plt.title(titles[i])
    plt.xticks([]), plt.yticks([])
plt.show()
```

效果对比：全局阈值在阴影处会丢失很多文字，自适应阈值能把整个数独清晰地二值化出来。

3. Otsu 阈值化（自动找最佳阈值）

问题：手动选阈值 127 很不科学，怎么自动找“最好”的阈值？
Otsu 方法：假设图像直方图是双峰的（前景和背景），它会自动计算类间方差最大的那个阈值。
使用方式：在 cv.threshold 中加上标志 cv.THRESH_OTSU，这时 thresh 参数可以随便写（通常写 0），函数会返回最优阈值。

```
img = cv.imread('noisy2.png', cv.IMREAD_GRAYSCALE)

# 1. 全局固定阈值
ret1, th1 = cv.threshold(img, 127, 255, cv.THRESH_BINARY)

# 2. Otsu 自动阈值
ret2, th2 = cv.threshold(img, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)

# 3. 先高斯滤波再 Otsu（效果更好）
blur = cv.GaussianBlur(img, (5, 5), 0)
ret3, th3 = cv.threshold(blur, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)

print("Otsu 自动找到的阈值:", ret2, ret3)   # 会打印类似 123.0 这样的值
```

小技巧：噪声大的图片先做高斯模糊（GaussianBlur），再用 Otsu，效果会显著提升。